Temario

I. BD Orientadas a Objetos

Tema 1. Bases de Datos Orientadas a Objetos

Tema 2. El modelo de clases de UML

Ejercicios de modelado conceptual OO

Tema 3. El modelo objeto-relacional

Prácticas de BDOR en Oracle

Tema 4. Diseño de BDOR

Ejercicios de diseño de BD (objeto-)relacionales con UML

II. BD Activas

Tema 5. Bases de Datos Activas

Tema 6. Disparadores en Oracle

Prácticas de Disparadores en Oracle

Modelo Objeto-Relacional: SQL:2003

Bibliografía

- **Tecnología y Diseño de Bases de Datos. Piattini, M., Marcos, E., Calero, C. y Vela, B. Ra-ma, 2006 (cap. 20).**
- Bases de Datos Objeto Relacionales. Marcos, E., Vela, B. y Vara J.M. Dickinson, Septiembre 2005.
- SQL: 2003 has been published. A. Eisenberg, J. Melton, K. Kulkarni, J. Michels, F. Zemke. ACM SIGMOD Record Volume 33 , Issue 1 (Marzo 2004).
- SQL:1999, formely known as SQL3. A. Eisemberg and J. Melton. SIGMOD Record, Vol. 28, No. 1, Marzo 1999, pp. 131-138.
- SQL: 1999. Understanding object-relational and other advances features. Jim Melton Morgan Kauffman, 2002.
- SQL99, SQL/MM, and SQLJ: An Overview of the SQL Standards. Nelson M. Mattos. IBM Database Common Technology, 1999.

Índice

1. Introducción y motivación
2. Large Objects Data Types (LOBs)
3. Tipos Definidos por el Usuario (UDTs)
    - 3.1. Tipos Distintos
    - 3.2. Tipos estructurados
4. Tipos referencia (REF)
5. Tipo ROW
6. Tipos Colección: Arrays y Multisets
7. Jerarquías de Tablas
8. Métodos
9. Productos: ORACLE

Principales Extensiones respecto al SQL-92

- Extensiones Objeto-Relacionales:
  - Tipos Definidos por el Usuario
  - Tipos fila (*ROW*) y referencia (*REF*)
  - Tipos colección (no 1FN)
  - Soporte de Objetos "grandes"
- Disparadores (*triggers)*
- Procedimientos Almacenados y Funciones Definidas por el Usuario
- Consultas recursivas
- Extensiones OLAP
- XML …

Beneficios de la Extensión Objeto-Relacional

- **Extensibilidad**: Capacidad de **ampliar el sistema de tipos** para dar soporte a las nuevas necesidades de las aplicaciones.
  - **Nuevos tipos de datos** que representen mejor el dominio de la aplicación
  - **Nuevas operaciones** para soportar el comportamiento de los tipos
- **Poder de expresión**: Necesidad de soportar **objetos y relaciones complejas.**
- **Reusabilidad**: Capacidad de compartir librerías de tipos existentes.
- **Integración**: de los modelos relacional y objetos en un solo lenguaje.
- **Nuevas consultas**: recursivas, multimedia, etc.

Principales extensiones para soporte de Objetos

- Large Objects (LOBs)
  - Binary
  - Character
- Tipos Definidos por el Usuario (UDTs)
  - Tipos Distintos
  - Tipos Estructurados
- Constructores de tipo
  - Tipos ROW
  - Tipos Referencia (REF)
- Tipos colección
  - Arrays
  - Multisets
- Métodos, funciones y procedimientos definidos por el usuario
- Jerarquías de tablas y de vistas

2. LOBs: Large OBjects

- Tipos de datos con **gran capacidad de almacenamiento** (hasta Gigabytes).
- Útiles para el almacenamiento de ***imágenes, sonido, texto formateado*** y otras necesidades ***multimedia*** de las aplicaciones actuales.
- Dos tipos LOB:
  - BLOB: Binary Large OBject (audio, imagen, vídeo)
  - CLOB: Character Large OBject (texto)
- Se mantienen **directamente** en la BD, **no** en ficheros externos.
- El tamaño se puede indicar en la definición (K, M o G).

```
CREATE TABLE libro (
        título          VARCHAR2(200),
        id_libro        INTEGER,
        resumen         CLOB (32K),
        texto_libro     CLOB (20M),
        película        BLOB (2G));
```

2. LOBs: Large OBjects

- Los tipos LOB pueden ser insertados, actualizados y recuperados como cualquier otro tipo
  - Debido al gran tamaño del tipo LOB, el SGBDOR precisa la definición de '***buffers***' del tamaño adecuado para su gestión, según sea cada LOB.
  - Los **LOCATORS** facilitan la manipulación de los LOBs grandes (GB) por las aplicaciones. Permiten su manipulación *sin tener que transferir el valor completo* a la aplicación.
- Los LOBs se **excluyen** de algunas operaciones
  - Greater Than (>) y Less Than (<)
  - Claves primarias (PK), claves ajenas (FK) y UNIQUE
  - GROUP BY y ORDER BY
  - Operadores: UNION, INTERSECT y EXCEPT
  - JOINS (como columnas del Join)
- Algunas operaciones soportadas por los LOBs:
  - Recuperar un valor (o parte de él)
  - Reemplazar un valor
  - Predicado LIKE
  - Concatenación
  - Funciones: SUBSTRING, LENGTH, TRIM, …

UDTs (User Defined Types)

- Definir la estructura de los datos y su comportamiento
  - Tipos Distintos
    - Fuerte tipado
    - Comportamiento
  - Tipos Estructurados
    - Fuerte tipado
    - Comportamiento
    - Encapsulación
    - Sustitutabilidad
    - Polimorfismo
    - Vinculación dinámica
    - Verificación de tipos estática

Tipos Distintos

En SQL-92 dos columnas de una tabla con distinta semántica podía estar definidas bajo el mismo tipo de datos

```
CREATE TABLE Sala (
        IdSala          CHAR (10),
        LongSala        INTEGER,
        AnchoSala       INTEGER,
        AreaSala        INTEGER,
        PerimSala       INTEGER));
```

```
UPDATE Sala
SET AreaSala=LongSala;
```

Semánticamente Incorrecto

¡¡¡Misma representación (INTEGER)
PERO distinto COMPORTAMIENTO!!!

Tipos Distintos

Dos tipos de datos tienen una misma representación pero distinto comportamiento

```
CREATE TYPE tipoSala
        AS CHAR(10) FINAL;

CREATE TYPE tipoMetros
        AS INTEGER FINAL;

CREATE TYPE tipoMetrosCuad
        AS INTEGER FINAL;

CREATE TABLE Sala (
        IdSala        tipoSala,
        LongSala      tipoMetros,
        AnchoSala     tipoMetros,
        AreaSala      tipoMetroscuad,
        PerimSala     tipoMetros));
```

```
UPDATE Sala
SET AreaSala=LongSala;
```

Error

```
UPDATE Sala
SET AnchoSala=LongSala;
```

Correcto

Tipos Distintos

- Se basa en el renombrado de tipos existentes:
  - Comparten la *misma representación*
  - Tienen *distinto comportamiento*
  - El tipo distinto no es comparable con el tipo fuente

- Operaciones definidas sobre los tipos distintos:
  - Operadores de comparación
  - Métodos y funciones
  - No **existe herencia** entre tipos distintos (*FINAL*)

Tipos Estructurados

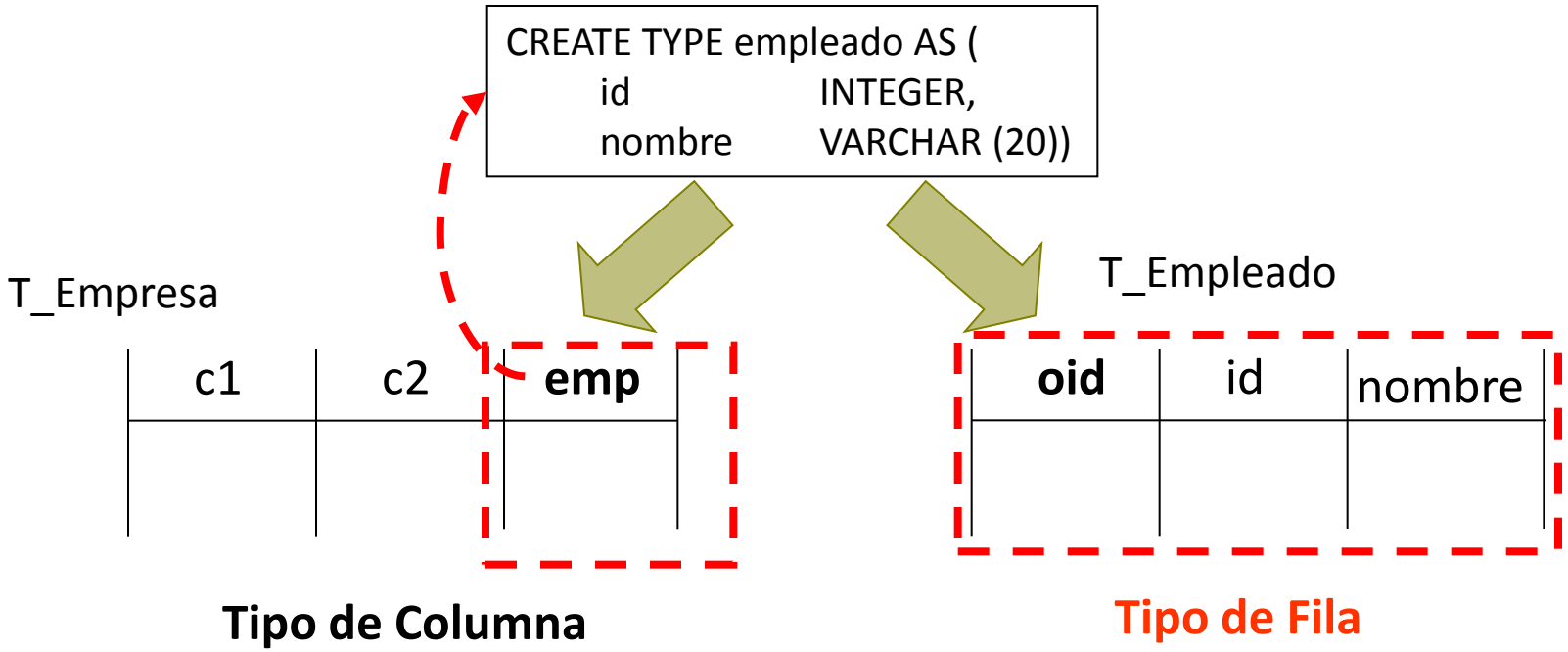
Se pueden usar como:

- **Tipo de columna**:
  - Tipo de datos valor (Tipo Valor)
  - Permiten modelar ***atributos*** de las entidades, como ***atributos compuestos***, sin existencia propia

    Ej: dirección (calle, ciudad, provincia)
  - Permiten mejorar el soporte para objetos multimedia (SQL/MM), como texto, imagen, vídeo, series de tiempo, punto, línea, etc.
- **Tipo de fila**:
  - Tipo de objeto, objetos que se desean que tengan existencia propia en la BD
  - Permiten modelar ***entidades*** con relaciones y comportamiento

    Ej: empleado, departamento, alumno...
  - Permiten mejorar el soporte para objetos de negocio, con relaciones y comportamiento

Tipos Estructurados



Tipos Estructurados

Se pueden usar en cualquier sitio donde se pueda usar un **tipo predefinido**:

- Atributos de otros tipos estructurados
- Parámetros de func., met. Y proc.
- Variables SQL
- Dominios o columnas

```
CREATE TYPE dirección AS (
        calle           CHAR (30),
        ciudad          CHAR (20),
        provincia       CHAR (2),
        codigoP         CHAR (5));

CREATE TYPE bitmap AS BLOB FINAL;

CREATE TYPE finca AS (
        propietario     REF (persona),
        precio          INTEGER,
        habitaciones    INTEGER,
        tamaño          DECIMAL (8,2),
        ubicación       dirección,
        imagen          bitmap);
```

También se pueden usar para definir **tablas y vistas**

```
CREATE TABLE propiedades OF finca;
```

Tablas Tipadas

Para que un tipo estructurado pase a ser un tipo de objeto es necesario definir una tabla sobre ese tipo → ***Tabla Tipada***

Intensión (estructura): definición de tipo estructurado

Extensión (tuplas/objetos): definición de tabla tipada

```
CREATE TYPE finca AS (
        precio                          INTEGER,
        Num_habitaciones INTEGER,
        tamaño                          DECIMAL (8,2),
        ubicación                       dirección);

CREATE TABLE propiedades OF finca;
```

Tipo Estructurado

Tabla Tipada

PROPIEDADES

REF	precio	num_habitaciones	tamaño	ubicación
				calle | ciudad | provincia | CP

OID

Atributos del tipo estructurado

4. Tipos Referencia

- Un tipo estructurado tiene un ***tipo referencia*** correspondiente (contructor de tipo).
- El tipo referencia contiene el valor del ***atributo REF*** de una fila de una tabla tipada.
- El tipo referencia puede ser usado donde cualquier otro tipo.

Tipos de Generación de OID:

- Generado por el sistema **REF IS SYSTEM GENERATED** (por defecto)

- Generado por el usuario **REF USING <tipo predefinido>**

```
CREATE TYPE propiedades as (.........)
NOT FINAL REF USING INTEGER;
```

- Derivada de un conjunto de atributos **REF (<atributos>)**

```
CREATE TYPE persona as (nss INTEGER, nombre CHAR(20), ...)
NOT FINAL REF (nss);
```

4. Tipos Referencia

El tipo Referencia permite implementar relaciones prescindiendo de claves ajenas.

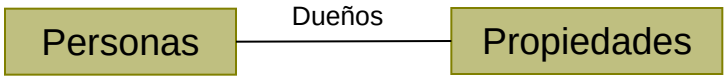
Las referencias <u>no tienen la misma semántica</u> que la restricción de integridad referencial:

- La integridad referencial implica una dependencia de inclusión (consistencia entre las tuplas de dos tablas) y las referencias no (pueden apuntar a ninguna parte).

  ⇨ referencia a **tipos de objetos** NO a tablas

- La integridad referencial no soporta la noción de fuerte tipado.

4. Tipos Referencia



- Path expression (referencias con ámbito) → <u>se eliminan los JOINS</u>

```
SELECT p.precio, p.dueño->nombre
FROM propiedades p
WHERE p.dueño->direccion.ciudad="Madrid";
```

- Las referencias se pueden usar para <u>invocar métodos</u>

```
SELECT p.precio, p.dueño->salario (2000)
FROM propiedades p;
```

- Las referencias se pueden usar para obtener el valor del tipo estructurado al que está referenciando (derreferencia)

SELECT p.precio, **DEREF** (p.dueño)
FROM propiedades p;

5. Tipo ROW

- Un tipo ROW es un tipo compuesto por varios atributos.
- Puede formar parte de un UDT, ser el tipo de una columna de una tabla o ser el tipo base de una tabla
- **No** se puede definir como tipo independiente (sólo anónimo).
  - Declaración implícita

```
CREATE TYPE empleado (
    id_emp  INTEGER,
    nombre  ROW (
                n_de_pila  VARCHAR(30),
                apellido   VARCHAR(30)),
    dirección ROW (
                calle      VARCHAR(50),
                ciudad     VARCHAR(30),
                provincia  CHAR(2)),
    salario  REAL );
    SELECT E.nombre.apellido
    FROM empleado E;
```

Array

- Conjunto de elemento homogéneos e indexados
- Los arrays permiten soportar columnas que contengan colecciones de elementos (**n1NF**)
- Se pueden definir sobre cualquier tipo de datos básico o UDT
- Características:
  - Diferencia entre la longitud máxima y la actual (como el CHAR VARYING)
  - Pueden definirse sobre cualquier tipo (excepto array)
  - Pueden definirse arrays en “cualquier sitio”
- Operaciones:
  - Acceso a los elementos por un número ordinal, Comparación, Constructores, Asignación, Concatenación, CAST, …
  - Facilidades de selección declarativa sobre arrays (por contenido o por posición, transformación implícita de un array en una tabla...)

6. Tipos colección: Array y Multiset

Ejemplo de Array

```
CREATE TYPE libro (
        título          VARCHAR (200),
        id_libro        INTEGER,
        autores         VARCHAR (15) ARRAY [20],
        resumen         CLOB (32K),
        texto_libro     CLOB (20M),
        película        BLOB (2G));
```

```
INSERT INTO libro (título, id_libro, autores)
        VALUES ("A guide to the SQL Standard", 15, ['Date', 'Darwen' ]);
```

```
SELECT id, autores[1] AS nombre
FROM libro;
```

Id_libro	nombre
15	Date

Multiset

- Un Multiset es un tipo colección de elementos homogéneos sin orden y que permite elementos repetidos.

- El tipo MULTISET también permite definir columnas que contengan colecciones de elementos y que, por lo tanto, no cumplen con la 1FN.

Existen numerosas funciones para operar con los Multisets:

- **MULTISET**() genera una colección vacia.
- **MULTISET**(<valores>) genera un multiset partiendo de una lista de valores.
- **MULTISET**(<SELECT>) convierte una relación en un multiset.
- **UNNEST**(<M>) convierte un multiset en una tabla virtual.
- **Element**(<M>) devuelve un elemento de un multiset M con un solo elemento.

…

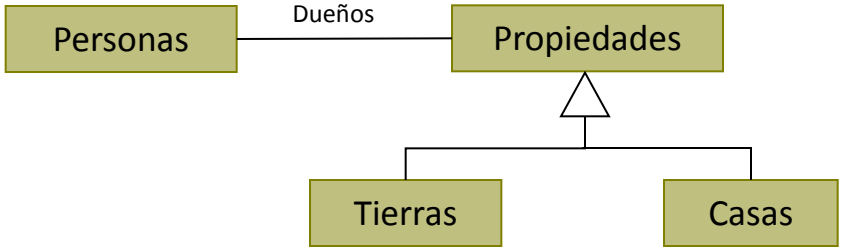
Ejemplo de MULTISET

```
CREATE TYPE Película (
título          VARCHAR (200),
id_película     INTEGER,
actores         VARCHAR (15) MULTISET,
resumen         CLOB (32K),
pelicula        BLOB (2G));
```

```
INSERT INTO Película (título,id_película,actores)
VALUES ("Top Gun", 3, Multiset ['Tom Cruise','Kelly McGillis','Val Kilmer']);
```

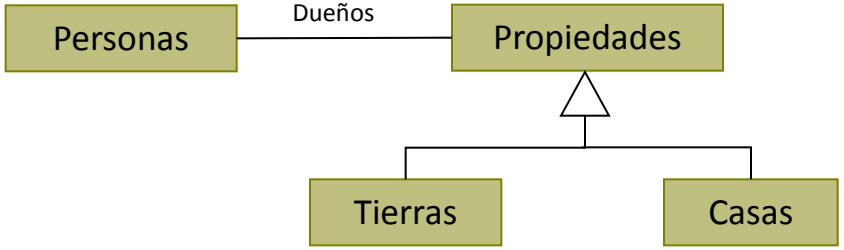
7. Jerarquías de Tablas

- SQL:2003 soporta herencia simple de tipos estructurados y de tablas tipadas (subtablas)
- Las subtablas heredan: atributos, restricciones, disparadores, métodos...




```
CREATE TYPE persona AS .... NOT FINAL;
CREATE TYPE propiedad AS (dueño REF (persona), ....) NOT FINAL;
CREATE TYPE tierra UNDER propiedad AS ( ....) NOT FINAL;
CREATE TYPE casa UNDER propiedad AS (...) NOT FINAL;

CREATE TABLE personas OF persona;
CREATE TABLE propiedades OF propiedad;
CREATE TABLE tierras OF tierra UNDER propiedades;
CREATE TABLE casas OF casa UNDER propiedades;
```

7. Jerarquías de Tablas



- Las consultas sobre la supertabla, devuelven también las filas de las subtablas

```
SELECT *
FROM propiedades
WHERE ....
```

} **Propiedades + Tierras + Casas**

- Se puede restringir las filas seleccionadas a las de la supertabla

```
SELECT *
FROM ONLY (propiedades)
WHERE ....
```

8. Métodos

Método: función SQL ligada a un UDT.

- Representan su comportamiento
- Se definen en el esquema
- La signatura se define separada de la especificación del cuerpo

```
CREATE TYPE empleado AS (
        id              INTEGER,
        nombre          VARCHAR (20),
        salario_base    DECIMAL (9,2),
        bono            DECIMAL (9,2))
        INSTANTIABLE NOT FINAL
        METHOD salario() RETURNS DECIMAL (9,2));

CREATE METHOD salario() FOR empleado
BEGIN
        ......
END;
```

Los métodos se **invocan** utilizando la notación “punto”:

```
SELECT mgr.salario()
FROM empleado;
```

8. Métodos

```
CREATE TYPE empleado AS (
        id                      INTEGER,
        …
        INSTANTIABLE NOT FINAL,
        METHOD salario() RETURNS DECIMAL (9,2);

CREATE TYPE manager UNDER empleado AS (
        stock_option        INTEGER)
        INSTANTIABLE NOT FINAL
        OVERRIDING METHOD salario() RETURNS DECIMAL (9,2),
        METHOD original1() RETURNS INTEGER);

CREATE METHOD salario() FOR empleado
BEGIN
        ......
END;
CREATE METHOD salario() FOR manager
BEGIN
        ......
END;
CREATE METHOD original1() FOR manager
BEGIN
        ......
END;
```

Dos tipos de métodos:

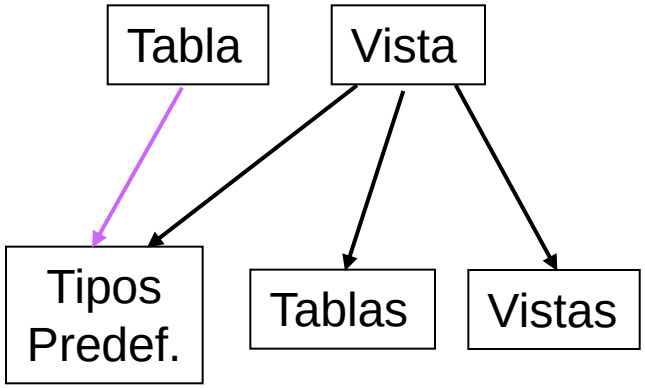
- **Originales** (del supertipo)
- **Overriding** (de los subtipos) → *polimorfismo*

Vinculación dinámica: los métodos sobrecargados se resuelven en tiempo de ejecución (el más específico).

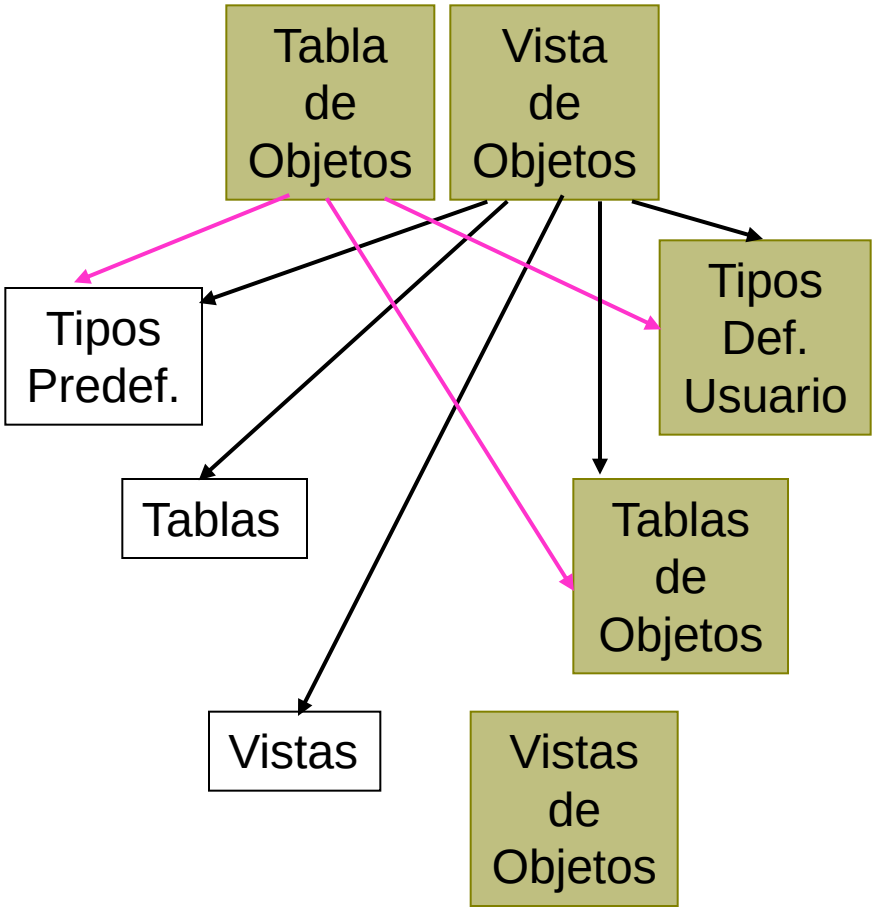
9. Productos

- DB2 Universal Database de IBM, http://www.ibm.com
- Universal Server de INFORMIX (ahora de IBM), http://www.informix.com
- INGRES II, Computer Associates, http://www.ingres.com
- **ORACLE** de Oracle Corporation, http://www.oracle.com
- SYBASE de SYBASE, http://www.sybase.com

Relacional



Objeto-Relacional



Ansteiy (1998)

9. Productos: Oracle

- Tipos predefinidos (Oracle built-in datatypes):
  - Carácter (CHAR, VARCHAR2, NCHAR, NVARCHAR2, LONG)
  - Numéricos (NUMBER, INTEGER, FLOAT)
  - DATE
  - ROWID
  - LOB (BLOB, CLOB, NCLOB, BFILE)
  - ....

LOBs (Large Objects): Propiedades

Tipos de datos con gran capacidad de almacenamiento (hasta 4 GigaBytes).

Se puede almacenar cualquier tipo de dato no estructurado y de gran tamaño

Tipos de LOBs:

- Almacenado Internamente (*Tablespaces*)
  - En la misma tabla (*in-line*)
  - Espacio de tabla distinto (*off-line*)
    - **BLOB:** Binary Large Object (grandes objetos binarios)
    - **CLOB**: Character Large Object (grandes objetos de textuales)
    - **NLOB**: National Character Large Object (vocabulario nacional)

- Almacenado Externamente
  - **BFILE**: Binary FILE (sólo lectura; ***referencia*** *a fichero externo gestionado por el Sistema Operativo*)

LOBs (Large Objects)

Ejemplo:

```
CREATE TABLE LIBRO (
   Titulo                    VARCHAR2(200),
   Id_Libro                  INTEGER,
   Resumen                   CLOB,
   Texto                     NLOB,
   Película          BFILE);

CREATE DIRECTORY dir_pelicula AS '/demo/libros';

INSERT INTO LIBRO (Titulo, Id_Libro, Película)
   VALUES ('Momo', 001, bfilename(dir_pelicula, 'momo.avi'));
```

9. Productos: Oracle

- Tipos definidos por el usuario:
  - Tipos Distintos (SQL:2003)
  - Tipos estructurados
- Constructores de Tipos:
  - Tipos de Objeto
  - Tipo REF
- Tipos Colección:
  - VARRAY
  - NESTED TABLES

Tipos estructurados

- Tipos Valor: tipo de una columna o de un atributo
- Tipos de Objeto: tipo de una tabla
- Se definen mediante la cláusula **CREATE TYPE <*nombretipo*> AS OBJECT**

```
CREATE  OR REPLACE TYPE dirección AS OBJECT(
        calle           VARCHAR2 (30),
        ciudad          VARCHAR2 (20),
        provincia       VARCHAR2 (2),
        codigoP         VARCHAR2 (5))
        /

CREATE TABLE persona (
        nombre          VARCHAR2(30),
        vive_en         dirección,
        foto            BLOB);
```

Tipo de dato valor (Columna)

Constructores de Tipos: Tipos Objeto

```
CREATE OR REPLACE TYPE Empleado AS OBJECT
      ( DNI                          NUMBER,
        Nombre                       VARCHAR2(30),
        Fecha_nac                    DATE,
        Pertenece_a  REF Departamento,
        MEMBER FUNCTION Edad RETURN NUMBER)
      /
```

```
CREATE TABLE Tabla_Empleado OF Empleado;
```

Tipo de objeto (Fila)

- **Intensión**: especificación del tipo estructurado
- **Extension**: especificación de la tabla tipada
- Se pueden crear **varias tablas tipadas** sobre un tipo estructurado
  - Una columna por cada atributo del tipo estructurado, además de REF.
- Las **restricciones** (clave primaria, unicidad) se definen sobre la tabla

Constructores de Tipos: Tipo Referencia

```
CREATE OR REPLACE TYPE Proyecto AS OBJECT
( ID_P              NUMBER,
  Nombre_P          VARCHAR2(30),
  FechaEntrega      Date)
/
CREATE OR REPLACE TYPE Jefe_Proyecto AS OBJECT
 (ID_Jefe           NUMBER,
  Nombre            VARCHAR2(30),
  Telefono          NUMBER,
  Dirige            REF Proyecto)
/
```

Referencia a un tipo de objeto

```
SELECT nombre, jp.dirige.nombre_p
FROM Jefe_Proyecto jp
WHERE ID_Jefe = 1256;
```

Tipos Colección: VARRAY

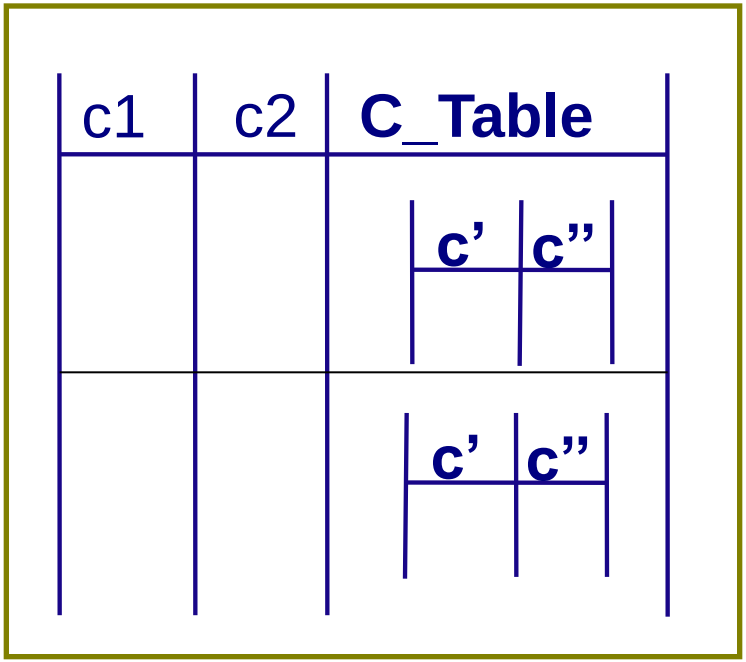
- Similar al ARRAY, difiere en la sintáxis de definición y su utilización

```
CREATE OR REPLACE TYPE TipoTelefono
            AS VARRAY(3) OF VARCHAR2(10)
/

CREATE TABLE Empleado
      ( DNI                    NUMBER,
        Nombre                 VARCHAR2(30),
        Teléfonos_contacto     TipoTelefono);

INSERT INTO Empleado VALUES ('9876543', 'Pepe',
            TipoTelefono('914445566', '606445566', '934445566'));
```

- No se puede acceder a los elementos individuales

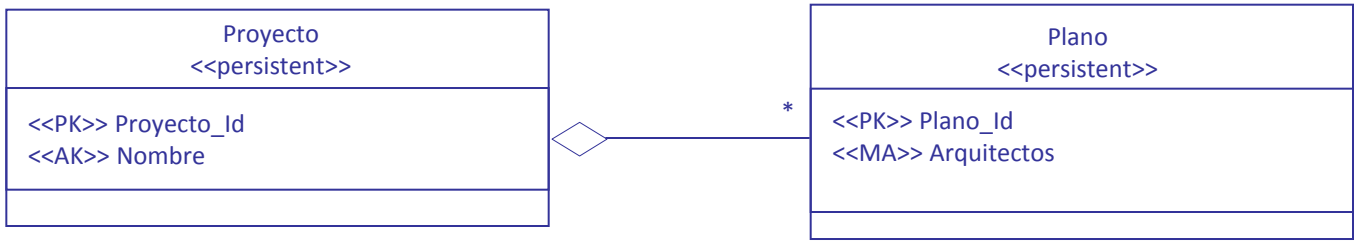
Tipos Colección: Nested Table

- Se corresponde con el MULTISET
- Es un tipo de datos definido como una tabla que puede utilizarce como el tipo de datos de una columna de otra tabla



- La coleccion de objetos, valores o referencias se almacenan en formato tabla.
- Se pueden recuperar de acuerdo al orden de indexación.
- Al definir una tabla tipada con un atributo de tipo tabla es necesario darle un nombre de almacenamiento a la Nested Table, mediante STORED AS.
- El acceso a los elementos de la Nested Table se hace desde la tabla principal

Tipos Colección: Nested Table




```
CREATE TYPE TipoPlano AS OBJECT
(Plano_ID          NUMBER,
Arquitectos        Tipo_Nombre)
/
CREATE TABLE Plano OF TipoPlano (PRIMARY KEY (Plano_Id));

CREATE OR REPLACE TYPE NT_Planos AS TABLE OF REF TipoPlano
/
CREATE TYPE TipoProyecto AS OBJECT
( Proyecto_Id      NUMBER,
  Nombre           VARCHAR2(30),
  Tiene_Plano      NT_Planos)
/
CREATE TABLE Proyecto OF TipoProyecto
(PRIMARY KEY (Proyecto_ID),
UNIQUE (Nombre))
NESTED TABLE Tiene_Plano STORE AS ListaPlanos;
```

Tipos Colección: Nested Table vs. VARRAY

	VARRAY	NESTED TABLE
Tamaño máximo	Sí	No
Borrado elementos ind.	No	Sí
Almacenamiento datos	In-line	Out-of-line
Matenimiento de orden	Sí	No

Ansteiy (1998)

Tipos Colección:
Nested Table vs. VARRAY

- Es importante el orden en el que se almacenan los elementos?
  - Si → VARRAY
- Se sabe de antemano el número de elementos de la colección?
  - Si → VARRAY
- Se necesitan consultas sobre la colección que nos permita tratar los elementos por separado?
  - Si → NESTED TABLE